



UNA PÁGINA DEMASIADO CONVINCENTE

CNO IV - SEGURIDAD INFORMÁTICA

MELANIE ALONDRA ALBA DIEZ
185583
ING. EN TECNOLOGÍAS DE LA INFORMACIÓN

Índice

Objetivo de la actividad	3
Descripción del escenario	3
Configuración del laboratorio	4
1. Diseño de la página web ficticia.....	4
2. Configuración del formulario HTML	4
3. Configuración de carpeta compartida en la máquina virtual	5
4. Transferencia y verificación del archivo en Kali Linux	6
5. Configuración de Social-Engineer Toolkit (SET)	7
6. Configuración de Credential Harvester	8
Evidencias y descripción del funcionamiento	9
1. Ejecución del servidor web local	9
2. Acceso al portal ficticio	10
3. Envío de datos de prueba	10
4. Registro de parámetros en SET	10
5. Generación y verificación del reporte XML	11
Explicación del flujo de captura de información	12
1. Funcionamiento del Credential Harvester	12
2. Recorrido de la información.....	12
3. Peticiones HTTP utilizadas	12
4. Servidor y registro de parámetros.....	13
5. Captura de credenciales y vulneración de una contraseña.....	13
6. Diagrama de flujo de la información	14
Indicadores de phishing	14
1. Solicitud inesperada de autenticación	14
2. Uso de urgencia o consecuencias para provocar una acción	14
3. Dirección web diferente a la esperada	14
4. Solicitud de información desde un enlace	14
5. Inconsistencias en el sitio o en la comunicación.....	14
Controles de seguridad propuestos	15
1. Autenticación multifactor resistente al phishing.....	15
2. Capacitación y concientización de los usuarios	15
3. Filtrado y autenticación del correo electrónico	15

4. Administradores de contraseñas y credenciales únicas	16
5. Monitoreo y respuesta ante accesos sospechosos	16
Conclusiones.....	16
Referencias	17

Objetivo de la actividad

El objetivo de esta actividad es implementar y analizar, dentro de un entorno de laboratorio controlado, una simulación de ingeniería social mediante **Social-Engineer Toolkit (SET)** y su módulo **Credential Harvester Attack Method**, utilizando una **página web institucional ficticia** diseñada específicamente para la práctica.

Mediante la actividad se busca comprender el funcionamiento de una técnica de captura de información utilizada en campañas de phishing, identificando el recorrido que realizan los datos desde que un usuario los introduce en un formulario web hasta que son recibidos y registrados por el servidor. Asimismo, se pretende reconocer los elementos visuales y técnicos que pueden hacer que una página fraudulenta resulte convincente, distinguir la captura de credenciales de la vulneración de una contraseña y analizar diferentes medidas de seguridad que pueden reducir el riesgo de este tipo de ataques.

Toda la práctica se realizó exclusivamente en un entorno local y controlado, utilizando datos ficticios y sin publicar el servidor hacia Internet ni utilizar páginas, dominios o credenciales pertenecientes a personas u organizaciones reales.

Descripción del escenario

Para la simulación se planteó un escenario en el que un usuario recibe un supuesto aviso relacionado con la expiración de su sesión institucional. Como parte del engaño, el usuario sería dirigido hacia un portal que aparenta corresponder al sistema de acceso de una institución, donde se le solicita introducir nuevamente su usuario y contraseña para continuar.

Para representar este escenario se diseñó una institución completamente ficticia denominada NovaTech Instituto Superior, así como un portal denominado Portal de Servicios Institucionales. La página presenta elementos propios de un sistema de autenticación, como identidad institucional, campos de usuario y contraseña, botón de inicio de sesión y un aviso indicando que la sesión del usuario ha finalizado por inactividad.

El mensaje de expiración representa el componente de **ingeniería social** del escenario, ya que busca proporcionar al usuario una razón aparentemente legítima para introducir nuevamente sus credenciales. En una campaña de phishing real, recursos de este tipo pueden aprovechar la confianza, la familiaridad o el sentido de urgencia para provocar una acción que el usuario normalmente no realizaría.

Sin embargo, el portal desarrollado para esta práctica no corresponde a ninguna institución existente y se utilizó únicamente dentro de la **máquina virtual Kali Linux**. De esta forma fue posible analizar el comportamiento técnico de la captura de información sin involucrar usuarios, credenciales, servicios o infraestructura externos.

Configuración del laboratorio

1. Diseño de la página web ficticia

La página utilizada para la simulación fue desarrollada inicialmente en el equipo anfitrión, utilizando **Visual Studio Code**. Se creó un único archivo denominado **index.html**, dentro del cual se integraron tanto la estructura HTML como los estilos CSS necesarios para la interfaz.

El diseño representa el portal de acceso de NovaTech Instituto Superior, una institución creada exclusivamente para esta actividad. Se incorporaron elementos que permitieran representar de manera realista un sistema institucional de autenticación: nombre e identidad visual del sistema, descripción del portal, aviso de acceso restringido, mensaje de sesión finalizada, campo para usuario institucional, campo para contraseña y botón de inicio de sesión.

La integración de todos los elementos de la página dentro de un solo archivo **index.html** facilitó posteriormente su importación en **SET** mediante la opción **Custom Import**.

Vista de la página web

2. Configuración del formulario HTML

Dentro del archivo **index.html** se creó un formulario destinado a representar el inicio de sesión. Para ello se utilizó el método HTTP **POST** y se establecieron los campos **username** y **password**.

La configuración principal del formulario fue la siguiente:

<form method="POST" action="/">

El campo correspondiente al usuario fue identificado mediante: **name="username"**

Mientras que el campo de contraseña utilizó: **name="password"**

Estos atributos son relevantes para el funcionamiento de la práctica debido a que permiten identificar los parámetros enviados por el navegador cuando el usuario presiona el botón **Iniciar sesión**. Posteriormente, **SET** analiza la información recibida y puede reconocer los campos enviados mediante la petición.

```
<form
  method="POST"
  action="/"
>

  <div class="form-group">

    <label for="username">
      Usuario institucional
    </label>

    <input
      type="text"
      id="username"
      name="username"
      placeholder="Ingresa tu usuario"
      autocomplete="off"
      required
    >

  </div>
```

```
<div class="form-group">

  <label for="password">
    Contraseña
  </label>

  <input
    type="password"
    id="password"
    name="password"
    placeholder="Ingresa tu contraseña"
    autocomplete="off"
    required
  >

</div>
```

```
<div class="form-options">

  <label class="remember">

    <input
      type="checkbox"
    >

    Mantener sesión iniciada
  </label>

</div>

<button
  type="submit"
  class="login-button"
>
  Iniciar sesión
</button>

</form>
```

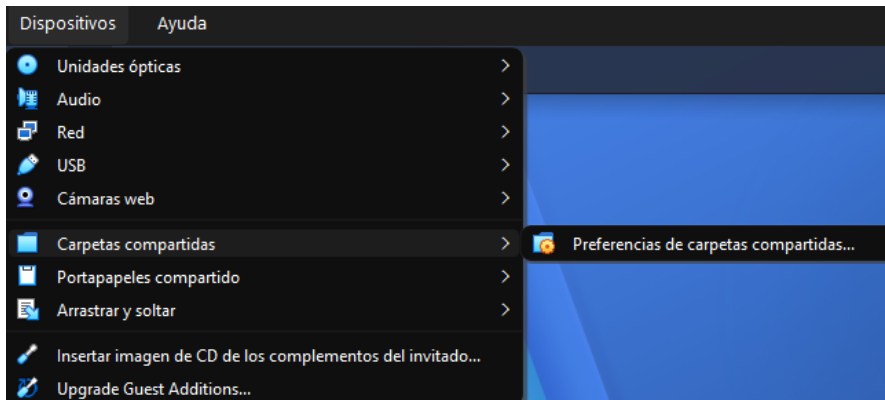
Código correspondiente al formulario

3. Configuración de carpeta compartida en la máquina virtual

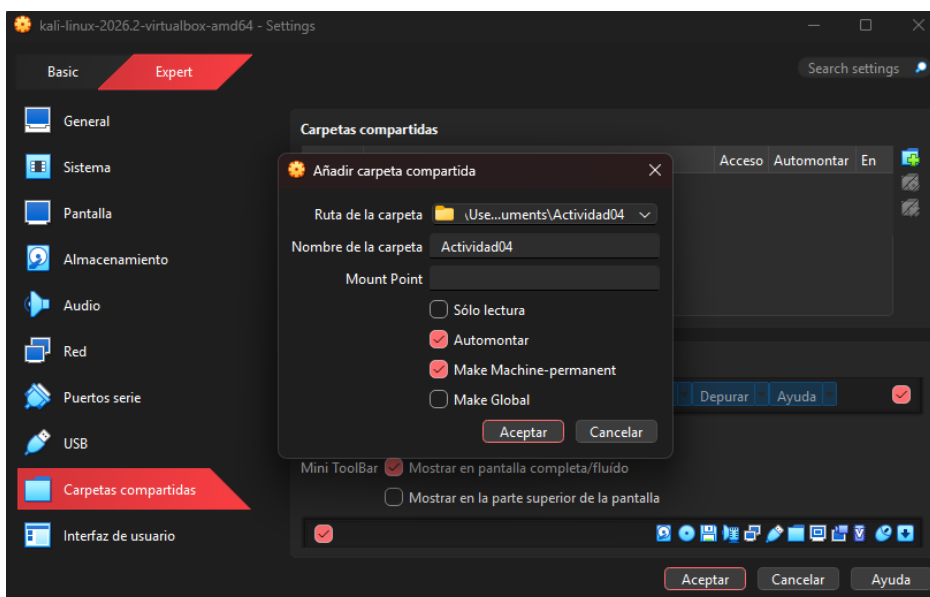
Debido a que el archivo **index.html** fue desarrollado originalmente en Windows, mientras que **SET** se ejecutaría dentro de la **máquina virtual Kali Linux**, fue necesario establecer un mecanismo para transferir el archivo entre ambos sistemas.

Para ello se utilizó la función de **Carpetas compartidas de VirtualBox**. Se seleccionó la carpeta **Actividad04**, ubicada en el equipo anfitrión, y se habilitaron las opciones **Automontar** y **Hacer permanente dentro de la máquina virtual**.

Esta configuración permitió que Kali Linux pudiera acceder al contenido de la carpeta sin necesidad de publicar los archivos en Internet o utilizar algún servicio externo.



Menú Dispositivos, donde se encuentra Carpetas compartidas



Configuración de la carpeta compartida.

4. Transferencia y verificación del archivo en Kali Linux

Se comprobó desde la terminal de Kali que la carpeta compartida se hubiera montado correctamente.

Para ello se ejecutó: ***ls /media/sf_Actividad04***

El resultado mostró el archivo: ***index.html***

Posteriormente se utilizó: ***pwd***

para comprobar el directorio de trabajo actual, obteniendo **/home/kali**. Estas verificaciones permitieron confirmar que Kali tenía acceso al archivo creado originalmente en Windows.

```
(kali㉿kali)-[~]  
$ ls /media/sf_Actividad04  
index.html  
  
(kali㉿kali)-[~]  
$ pwd  
/home/kali
```

Terminal máquina virtual Kali Linux

Posteriormente se creó dentro del directorio personal de Kali una carpeta denominada **Actividad04**. Se ingresó a ella y se copió el archivo **index.html** desde la carpeta compartida. Finalmente, mediante **ls**, se verificó que el archivo estuviera disponible en el nuevo directorio.

```
(kali㉿kali)-[~]  
$ mkdir "Actividad04"  
  
(kali㉿kali)-[~]  
$ cd "Actividad04"  
  
(kali㉿kali)-[~/Actividad04]  
$ cp /media/sf_Actividad04/index.html ~/Actividad04  
  
(kali㉿kali)-[~/Actividad04]  
$ ls  
index.html
```

Terminal máquina virtual Kali Linux

De esta manera, el sitio quedó disponible en la ruta: **/home/kali/Actividad04**

5. Configuración de Social-Engineer Toolkit (SET)

Una vez preparado el archivo de la página web, se inició Social-Engineer Toolkit mediante: **sudo setoolkit**.

SET solicitó los privilegios correspondientes y posteriormente mostró su menú principal.

```
Select from the menu:  
  
1) Social-Engineering Attacks  
2) Penetration Testing (Fast-Track)  
3) Third Party Modules  
4) Update the Social-Engineer Toolkit  
5) Update SET configuration  
6) Help, Credits, and About
```

En primer lugar, se seleccionó: **1) Social-Engineering Attacks**

Esta categoría reúne diferentes vectores relacionados con técnicas de ingeniería social.


```
Select from the menu:

1) Spear-Phishing Attack Vectors
2) Website Attack Vectors
3) Infectious Media Generator
4) Create a Payload and Listener
5) Mass Mailer Attack
6) Arduino-Based Attack Vector
7) Wireless Access Point Attack Vector
8) QRCode Generator Attack Vector
9) Powershell Attack Vectors
10) Third Party Modules
```

Dentro del siguiente menú se seleccionó: **2) Website Attack Vectors**

Esta opción permite trabajar con escenarios de ingeniería social basados en sitios web.

```
1) Java Applet Attack Method
2) Metasploit Browser Exploit Method
3) Credential Harvester Attack Method
4) Tabnabbing Attack Method
5) Web Jacking Attack Method
6) Multi-Attack Web Method
7) HTA Attack Method
```

Posteriormente se seleccionó: **3) Credential Harvester Attack Method**

```
1) Web Templates
2) Site Cloner
3) Custom Import
```

y, dentro de sus opciones: **3) Custom Import**

La elección de **Custom Import** permitió utilizar la página *index.html* desarrollada específicamente para esta práctica, en lugar de emplear una plantilla o clonar un sitio existente. Esta decisión también permitió respetar las restricciones del laboratorio, ya que no se utilizó la identidad ni el portal de una organización real.

6. Configuración de Credential Harvester

Un **Credential Harvester** es un mecanismo diseñado para recibir y registrar información introducida en formularios web, especialmente campos asociados con procesos de autenticación. Dentro de una campaña de phishing, su función consiste en actuar como el componente que recibe los datos que una persona entrega a una página fraudulenta después de haber sido persuadida para interactuar con ella.

En esta práctica, **SET** permitió simular dicho comportamiento exclusivamente con datos ficticios y dentro del entorno local.

Al configurar el módulo se estableció la dirección: **127.0.0.1**

Esta dirección corresponde a **localhost**, es decir, a la propia máquina donde se ejecuta el servicio. Por lo tanto, Kali Linux actuó simultáneamente como el equipo que ejecutaba SET y como servidor web local. La utilización de 127.0.0.1 permitió mantener la simulación aislada y evitar la publicación del servidor hacia Internet.

Posteriormente se indicó como ubicación del sitio: **/home/kali/Actividad04**

SET encontró el archivo index.html y se seleccionó la opción para copiar únicamente dicho archivo. Finalmente, se estableció como URL: **http://127.0.0.1**

Tras completar la configuración, SET indicó que **Credential Harvester estaba ejecutándose en el puerto 80** y que mostraría en la terminal la información recibida. Estos valores coinciden con la configuración registrada durante la práctica.

```
set:webattack> IP address for the POST back in Harvester/Tabnabbing [10.0.2.15]: 127.0.0.1
[!] Example: /home/website/ (make sure you end with /)
[!] Also note that there MUST be an index.html in the folder you point to.
set:webattack> Path to the website to be cloned: /home/kali/Actividad04
[*] Index.html found. Do you want to copy the entire folder or just index.html?

1. Copy just the index.html
2. Copy the entire folder

Enter choice [1/2]: 1
[-] Example: http://www.blah.com
set:webattack> URL of the website you imported: http://127.0.0.1

The best way to use this attack is if username and password form fields are available. Regardless, this captures all POSTs on a website.
[*] The Social-Engineer Toolkit Credential Harvester Attack
[*] Credential Harvester is running on port 80
[*] Information will be displayed to you as it arrives below:
```

Evidencias y descripción del funcionamiento

1. Ejecución del servidor web local

Una vez finalizada la configuración, SET inició el servicio web necesario para presentar la página mediante el puerto 80.

Desde una segunda terminal se ejecutó Firefox utilizando: **firefox http://127.0.0.1**

De esta manera, el navegador solicitó al servidor local la página configurada anteriormente.

La terminal de SET registró una solicitud **GET / HTTP/1.1** con código de respuesta **200**, lo cual permitió comprobar que el servidor recibió correctamente la petición y pudo entregar el recurso solicitado.

```
127.0.0.1 - - [31/Aug/2026 04:30:22] "GET / HTTP/1.1" 200 -
```

2. Acceso al portal ficticio

Al ingresar a **http://127.0.0.1** desde Firefox se mostró el portal ficticio de NovaTech previamente desarrollado.

A diferencia de abrir directamente el archivo **index.html**, en este punto el navegador ya estaba solicitando el contenido a un servidor web local, representado por **SET**. Esto permitió que las interacciones realizadas con el formulario pudieran ser recibidas y analizadas por **Credential Harvester**.

3. Envío de datos de prueba

Para comprobar el funcionamiento del formulario se introdujeron exclusivamente credenciales ficticias. Entre las pruebas realizadas se utilizaron los usuarios:

usuario1

usuario2

usuario3

junto con una contraseña ficticia de prueba.

Al presionar **Iniciar sesión**, el navegador envió los valores correspondientes a los campos **username** y **password** hacia el servidor local.

Usuario institucional

usuario1

Contraseña

.....

☐ Mantener sesión iniciada

Iniciar sesión

4. Registro de parámetros en SET

Después del envío del formulario, **SET** analizó la información recibida e identificó los campos correspondientes al usuario y la contraseña.

En la terminal se mostraron mensajes como:

POSSIBLE USERNAME FIELD FOUND

POSSIBLE PASSWORD FIELD FOUND

junto con los parámetros enviados. La prueba se repitió con usuario1, usuario2 y usuario3, permitiendo comprobar que cada interacción generaba un nuevo registro.

```

127.0.0.1 - - [31/Aug/2026 04:30:22] "GET / HTTP/1.1" 200 -
[*] WE GOT A HIT! Printing the output:
POSSIBLE USERNAME FIELD FOUND: username=usuario1
POSSIBLE PASSWORD FIELD FOUND: password=contrasena
[*] WHEN YOU'RE FINISHED, HIT CONTROL-C TO GENERATE A REPORT.

127.0.0.1 - - [31/Aug/2026 04:34:59] "GET / HTTP/1.1" 200 -
[*] WE GOT A HIT! Printing the output:
POSSIBLE USERNAME FIELD FOUND: username=usuario2
POSSIBLE PASSWORD FIELD FOUND: password=contrasena
[*] WHEN YOU'RE FINISHED, HIT CONTROL-C TO GENERATE A REPORT.

127.0.0.1 - - [31/Aug/2026 04:37:31] "GET / HTTP/1.1" 200 -
[*] WE GOT A HIT! Printing the output:
POSSIBLE USERNAME FIELD FOUND: username=usuario3
POSSIBLE PASSWORD FIELD FOUND: password=contrasena
[*] WHEN YOU'RE FINISHED, HIT CONTROL-C TO GENERATE A REPORT.

```

Este resultado demuestra el funcionamiento del **Credential Harvester**: SET no necesitó conocer previamente los datos utilizados, sino que los recibió porque el formulario los transmitió al servidor después de que el usuario los introdujera voluntariamente.

5. Generación y verificación del reporte XML

Al finalizar la simulación se utilizó **Ctrl + C** para detener el servicio. SET informó que había exportado los resultados obtenidos a un archivo en formato XML dentro de:

/root/.set/reports/

Posteriormente se accedió desde la terminal al directorio correspondiente y se abrió el archivo generado. En su contenido fue posible identificar los parámetros correspondientes a las tres pruebas realizadas: usuario1, usuario2 y usuario3, junto con la contraseña ficticia introducida durante la simulación.

```

(kali@kali)-[~]
$ sudo ls -lt /root/.set/reports/
[sudo] password for kali:
ls: cannot access '-': No such file or directory
ls: cannot access 'lt': No such file or directory
/root/.set/reports/:
'2026-08-31 04:39:15.864880.xml'  files

(kali@kali)-[~]
$ sudo cat '/root/.set/reports/2026-08-31 04:39:15.864880.xml'
<?xml version="1.0" encoding='UTF-8'?>
<harvester>
  URL=http://127.0.0.1
  <url>      <param>username=usuario1</param>
             <param>password=contrasena</param>
  </url>
  <url>      <param>username=usuario2</param>
             <param>password=contrasena</param>
  </url>
  <url>      <param>username=usuario3</param>
             <param>password=contrasena</param>
  </url>
</harvester>

```

La existencia de este archivo demuestra que la información no solamente fue mostrada temporalmente en la terminal, sino que **SET** también pudo conservar un registro de los parámetros recibidos durante la ejecución.

Explicación del flujo de captura de información

1. Funcionamiento del Credential Harvester

Credential Harvester representa la parte del escenario encargada de recibir y registrar los datos enviados desde un formulario web.

Dentro de una campaña real de phishing, una página fraudulenta puede diseñarse para parecer legítima y persuadir a una persona para que introduzca información sensible. Cuando la persona envía el formulario, los datos no necesariamente llegan al servicio que cree estar utilizando, sino a infraestructura controlada por quien creó la página.

En el laboratorio realizado, este comportamiento se reprodujo de manera segura utilizando únicamente **127.0.0.1**, una institución ficticia y datos de prueba.

2. Recorrido de la información

El proceso comienza cuando el usuario accede desde el navegador al portal alojado por el servidor local. El servidor responde entregando el contenido del **index.html**, que el navegador interpreta y representa gráficamente.

Posteriormente, el usuario introduce información en los campos **username** y **password**. Estos datos permanecen inicialmente en el formulario del navegador.

Cuando se presiona **Iniciar sesión**, el navegador construye una **solicitud HTTP** para enviar los parámetros al servidor especificado por el formulario. Debido a que el formulario fue configurado con `method="POST"`, los valores son enviados como parte de la solicitud HTTP hacia el servidor.

SET, que se encuentra ejecutando **Credential Harvester**, recibe dicha petición, analiza los parámetros enviados e identifica los campos asociados al usuario y la contraseña. Finalmente, presenta la información encontrada en la terminal y puede conservarla en el **reporte XML**.

3. Peticiones HTTP utilizadas

Durante la práctica pueden distinguirse conceptualmente dos momentos.

Primero, el navegador necesita obtener la página del servidor. Por ello se observó en SET una solicitud: **GET / HTTP/1.1**

con una respuesta: **200**

El método GET corresponde en este caso a la solicitud del recurso web, mientras que 200 indica que el servidor pudo atender correctamente la solicitud.

Posteriormente ocurre el envío del formulario. Al estar configurado mediante:

<form method="POST" action="/">

el navegador transmite al servidor los parámetros introducidos por el usuario. SET procesa esta información e identifica los valores asociados a:

username

password

Por lo tanto, es importante distinguir entre la solicitud utilizada para obtener la página y la petición generada al enviar la información del formulario.

4. Servidor y registro de parámetros

En esta práctica, SET actuó como servidor local utilizando el **puerto 80**, mientras que **127.0.0.1** hizo que toda la comunicación permaneciera dentro de la misma máquina virtual.

Cuando **Credential Harvester** recibe la información del formulario, analiza los nombres de los parámetros y reconoce aquellos que pueden corresponder a credenciales. Esto explica los mensajes **POSSIBLE USERNAME FIELD FOUND** y **POSSIBLE PASSWORD FIELD FOUND** observados durante la ejecución.

Posteriormente, SET permitió exportar los datos recibidos a un **archivo XML**. La inspección de dicho archivo confirmó la existencia de los tres conjuntos de parámetros utilizados durante las pruebas.

5. Captura de credenciales y vulneración de una contraseña

Es importante señalar que capturar una contraseña no significa vulnerarla o descifrarla.

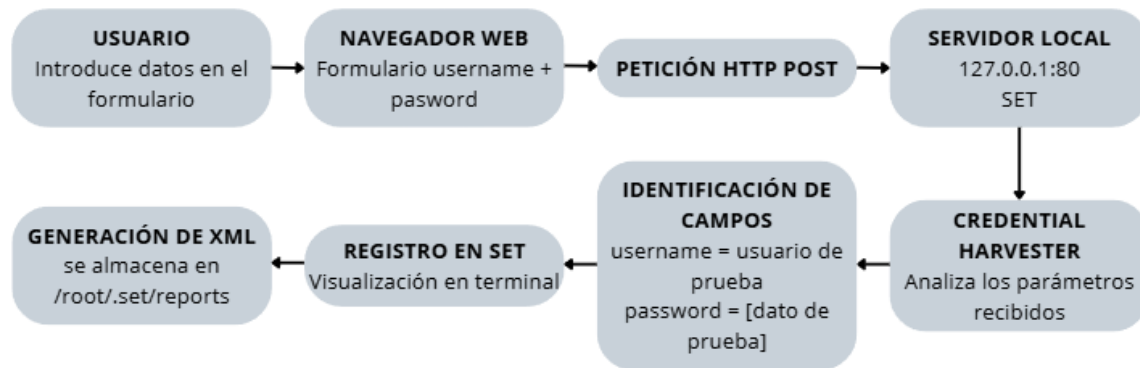
En esta práctica no se realizó fuerza bruta, explotación criptográfica, descifrado ni ningún procedimiento destinado a descubrir una contraseña desconocida. Los valores aparecieron en SET porque fueron introducidos directamente en el formulario y posteriormente transmitidos al servidor.

Captura de credenciales: la persona proporciona voluntariamente un usuario y una contraseña y el sistema fraudulento recibe esos valores.

Vulneración o cracking de una contraseña: implica intentar obtener o descubrir una contraseña que el atacante no conoce mediante técnicas diferentes, por ejemplo, aprovechando contraseñas débiles, hashes comprometidos o intentos automatizados.

La diferencia resulta fundamental para comprender el riesgo de la ingeniería social: en un ataque de phishing exitoso, el atacante puede no necesitar vulnerar técnicamente la contraseña si consigue convencer al propio usuario de entregarla.

6. Diagrama de flujo de la información



Indicadores de phishing

La simulación permitió identificar diferentes elementos que, en un escenario real, podrían funcionar como indicadores de que una página o mensaje forma parte de una campaña de phishing.

1. Solicitud inesperada de autenticación

Un primer indicador es que se solicite iniciar sesión nuevamente sin que el usuario haya realizado una acción que justifique dicha autenticación. Un aviso inesperado relacionado con la cuenta debe motivar al usuario a verificar directamente el servicio antes de introducir sus credenciales.

2. Uso de urgencia o consecuencias para provocar una acción

Los mensajes relacionados con sesiones expiradas, cuentas bloqueadas, accesos sospechosos o pérdida de servicios pueden generar presión para actuar rápidamente. La urgencia es una técnica habitual de ingeniería social porque puede reducir el tiempo que una persona dedica a verificar la legitimidad de una solicitud.

3. Dirección web diferente a la esperada

Aunque una página tenga una apariencia convincente, su dirección puede revelar que no pertenece al servicio que intenta representar. Por esta razón, es importante verificar cuidadosamente el dominio antes de introducir información sensible. En el laboratorio, 127.0.0.1 evidencia inmediatamente que se trata de un servidor local; en un ataque real podrían utilizarse dominios visualmente similares o engañosos.

4. Solicitud de información desde un enlace

Recibir un mensaje que dirige hacia una página y posteriormente solicita usuario, contraseña u otra información sensible constituye una señal que debe verificarse. En lugar de autenticarse desde el enlace recibido, el usuario puede acceder directamente al servicio mediante una dirección conocida o aplicación oficial.

5. Inconsistencias en el sitio o en la comunicación

Errores de redacción, enlaces que no funcionan, diferencias visuales, información de contacto extraña, elementos fuera de lugar o comportamientos diferentes a los del portal

habitual pueden revelar una página fraudulenta. Sin embargo, este indicador por sí solo no es suficiente, ya que una página de phishing técnicamente bien elaborada puede reproducir de manera muy convincente una interfaz legítima.

Controles de seguridad propuestos

1. Autenticación multifactor resistente al phishing

La implementación de **autenticación multifactor (MFA)** agrega requisitos adicionales al uso de una contraseña. De esta forma, la exposición de una contraseña no necesariamente proporciona acceso inmediato a la cuenta. CISA señala que MFA incrementa la protección incluso cuando una contraseña ha sido comprometida y recomienda especialmente mecanismos resistentes al phishing para cuentas sensibles.

Alcance: reduce considerablemente el riesgo de acceso no autorizado cuando una contraseña ha sido capturada.

Limitación: no elimina la ingeniería social y algunos métodos tradicionales de MFA también pueden ser objeto de engaño. Por ello deben preferirse mecanismos resistentes al phishing cuando sea posible.

2. Capacitación y concientización de los usuarios

Las organizaciones deben capacitar periódicamente a sus usuarios para reconocer mensajes sospechosos, revisar remitentes y dominios, evitar autenticarse mediante enlaces inesperados y reportar posibles intentos de phishing.

Alcance: actúa directamente sobre el componente humano explotado por la ingeniería social y puede ayudar a detectar campañas que superen controles técnicos.

Limitación: depende del comportamiento humano. Incluso una persona capacitada puede cometer errores ante campañas especialmente convincentes, situaciones de presión o ataques personalizados.

3. Filtrado y autenticación del correo electrónico

Los sistemas de correo pueden implementar filtros antiphishing, análisis de enlaces y archivos, así como mecanismos de autenticación de dominio como SPF, DKIM y DMARC. Estos controles permiten reducir la cantidad de mensajes fraudulentos que llegan a los usuarios.

Alcance: puede bloquear una parte importante de los intentos antes de que el usuario tenga contacto con ellos.

Limitación: ningún filtro detecta el 100 % de los mensajes maliciosos. Los atacantes pueden utilizar dominios nuevos, cuentas legítimas comprometidas o técnicas que inicialmente no sean reconocidas por los sistemas automáticos.

4. Administradores de contraseñas y credenciales únicas

El uso de un administrador de contraseñas permite mantener contraseñas diferentes y robustas para cada servicio. Además, algunos administradores asocian las credenciales con el dominio correspondiente, lo que puede proporcionar una señal adicional cuando el usuario se encuentra en una página distinta.

Alcance: reduce especialmente el impacto posterior de una credencial comprometida, ya que evita que la misma contraseña pueda reutilizarse automáticamente contra múltiples servicios. OWASP señala que la reutilización de credenciales permite que pares de usuario y contraseña obtenidos mediante filtraciones o phishing sean probados contra otros sitios.

Limitación: no evita por sí mismo que una persona introduzca manualmente una contraseña en una página fraudulenta y requiere buenas prácticas para proteger el propio administrador.

5. Monitoreo y respuesta ante accesos sospechosos

Las organizaciones pueden supervisar intentos de autenticación y generar alertas ante comportamientos anómalos, como accesos desde ubicaciones o dispositivos inusuales, múltiples intentos fallidos o patrones diferentes al comportamiento normal de una cuenta.

Alcance: permite detectar y responder ante posibles usos de credenciales comprometidas, incluso después de que un ataque de phishing haya tenido éxito. También facilita revocar sesiones, restablecer credenciales y analizar el incidente.

Limitación: generalmente actúa después o durante el intento de utilización de las credenciales y puede producir falsos positivos. Por ello no sustituye los controles preventivos, sino que debe formar parte de una estrategia de defensa en profundidad.

Conclusiones

La realización de esta práctica permitió comprender de manera aplicada cómo una técnica de ingeniería social puede combinar elementos visuales y técnicos para obtener información proporcionada por un usuario. Mediante una página institucional ficticia y el módulo Credential Harvester de Social-Engineer Toolkit fue posible observar el recorrido completo de los datos: desde su introducción en el formulario, su transmisión mediante una petición HTTP hacia el servidor local, su identificación como parámetros y finalmente su registro por SET.

Uno de los aspectos más importantes observados es que este tipo de escenario no depende necesariamente de vulnerar técnicamente una contraseña. En la simulación, SET no descifró, adivinó ni rompió ninguna clave; simplemente recibió los datos que fueron enviados mediante el formulario. Esto demuestra por qué la ingeniería social representa un riesgo significativo: incluso cuando los sistemas tecnológicos cuentan con mecanismos de seguridad adecuados, una persona puede ser persuadida para proporcionar información sensible voluntariamente.

El desarrollo del portal ficticio también permitió observar que una página visualmente organizada y aparentemente institucional puede generar una sensación de legitimidad. Por esta razón, identificar un intento de phishing no debe depender únicamente de que el sitio tenga errores evidentes de diseño. Es necesario analizar otros elementos, como el origen del mensaje, el dominio utilizado, la solicitud inesperada de información, el contexto y cualquier presión para actuar rápidamente.

Finalmente, la actividad permitió concluir que la prevención del phishing requiere combinar diferentes medidas. La capacitación de los usuarios es importante, pero debe complementarse con controles técnicos como autenticación multifactor resistente al phishing, filtrado de comunicaciones, contraseñas únicas y mecanismos de monitoreo y respuesta. Ninguna medida individual elimina completamente el riesgo; la protección más efectiva surge de utilizar varias capas de seguridad que reduzcan tanto la probabilidad de que el engaño tenga éxito como sus posibles consecuencias.

Referencias

Cybersecurity and Infrastructure Security Agency. (2024). *Phishing: Simple tips.*

[CISA — Phishing: Simple Tips](#)

Cybersecurity and Infrastructure Security Agency. *Multi-factor authentication.*

[CISA — Multi-Factor Authentication](#)

Cybersecurity and Infrastructure Security Agency, National Security Agency, Federal Bureau of Investigation, & Multi-State Information Sharing and Analysis Center. (2025). *Phishing guidance: Stopping the attack cycle at phase one.*

[CISA — Phishing Guidance](#)

OWASP Foundation. *Credential stuffing.*

[OWASP — Credential Stuffing](#)

OWASP Foundation. *Credential Stuffing Prevention Cheat Sheet.*

[OWASP — Credential Stuffing Prevention](#)